



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

One-Class Centered Ellipsoidal Support Vector Machines for Wireless Sensor Networks

In partial fulfillment of BA in Mathematics
in the School of Mathematics

Name: Grace Madison
Student ID: 22303044
Supervision: Prof. Kirk Soodhalter

Date of Submission: 7 April, 2026
Project Code: <https://github.com/graceannmad/TCDcapstone-SVM>



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

Declaration concerning plagiarism

I hereby declare that this thesis is my own work where appropriate citations have been given and that it has not been submitted as an exercise for a degree at this or any other university.

I have read and I understand the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at <http://www.tcd.ie/calendar>.

Name: Grace Madison

Student ID: 22303044

Signature: *Grace Madison*

Date: 7 April, 2026

ABSTRACT

Wireless sensor networks are commonly used to monitor environmental data and often benefit from online anomaly detection systems. Anomaly detection for WSNs can help avoid malfunction and allow for quick data interpretation. We will be looking at a specific framework for anomaly detection throughout this report: the Support Vector Machine (SVM). We develop the theory behind SVMs from the hard-margin classifier to the more standard soft-margin formulation. We including the construction of SVMs and how to specify them to anomaly detection tasks. We look at three variants of SVMs: the One-Class SVM (OC-SVM), the Ellipsoidal SVM (e-SVM), and the One-Class Centered Ellipsoidal SVM (OC-CESVM). Throughout the report, we build up to the OC-CESVM scheme in tandem with a variant of Principal Component Analysis (PCA) that is proposed by [16] as an effective and lightweight anomaly detection scheme for WSNs.

Keywords: support vector machine, anomaly detection, wireless sensor networks.

CONTENTS

1	Introduction	1
2	Standard Support Vector Machines	2
2.1	Linearly Separable Data	2
2.2	Non-Linear Classification	7
2.3	Sequential Minimal Optimization (SMO) Algorithm	15
3	Variations of the Support Vector Machine	19
3.1	One-Class Support Vector Machines	19
3.2	Ellipsoidal Support Vector Machines	22
3.3	One-Class Centered Ellipsoidal Support Vector Machines	23
3.4	Variations on the SMO Algorithm	26
4	Data Preparation	27
4.1	Normalization and Data Cleaning	27
4.2	Principal Component Analysis and Variations	28
5	Future Work	28
	Appendix	30
	References	38

1 INTRODUCTION

BACKGROUND

Throughout this project, we will be looking at the Support Vector Machine (SVM), a machine learning algorithm that is used for data classification. SVMs have wide ranging applications in data classification as well as anomaly classification. We investigate the system described in [16] which is a variation of a SVM with Principal Component Analysis (PCA). Specifically, it is a Centered Ellipsoidal One-Class Support Vector Machine with Candid Covariance Free Incremental Principal Component Analysis. They refer to this scheme as a whole as CESVM-DR where DR means “dimension reduction.”

This system is used for anomaly detection in Wireless Sensor Networks (WSNs). A common type of WSN are climate monitoring systems that measure things like temperature, humidity, and light. WSNs monitoring environmental factors like weather are typically situated in extreme locations like mountain ranges or forests. Other applications of WSNs include in healthcare, smart buildings, and vehicle tracking for traffic monitoring [4]. In most situations, WSNs must be monitored for sensor failure or malfunction, outside tampering like malicious behavior, or rare/critical events. In order to identify these events efficiently and without needing to send large packages of monitored data to outside computers, a SVM can be used. Once trained, SVMs provide a lightweight online anomaly detection framework that can be easily applied in potentially remote or resource constrained environments.

As a data classification algorithm, SVMs can be easily adjusted to train on one “class” or type of data that corresponds with normal behavior and then to detect data points that fall outside of this established “normal” which are referred to as anomalies. This is the type of SVM we will be looking at in relation to WSN monitoring in this report.

OVERALL SUMMARY

In Section 2 we develop the standard SVM by consulting two textbooks: [2] and [6]. We detail the mathematical development of SVMs from defining optimal hyperplanes through a discussion of various kernels for SVMs. The algorithm used to train SVMs, the Sequential Minimal Optimization (SMO) algorithm, is also derived and implemented.

Section 3 details variations on the standard SVM scheme including One-Class SVMs, Ellipsoidal SVMs, and the combined One-Class Centered Ellipsoidal SVM. We also detail the modifications of the SMO algorithm required to train these SVM variations.

In Section 4 we discuss the pre-processing required for dataset from the Lausanne Urban Canopy Experiment from 2007 (LUCE) [8]. This dataset is commonly used in WSN research and is used in [16]. We include normalization, Principal Component Analysis (PCA), and Candid Covariance-Free Incremental Principal Component Analysis (CCIPCA).

Finally, Section 5 proposes continued related work and potential future applications for this type of anomaly detection in different contexts.

All images in this report were generated by myself using code available in my [GitHub](#). Code appearing in the Appendix is also available in this GitHub repository.

2 STANDARD SUPPORT VECTOR MACHINES

The fundamental goal of a SVM is to find a decision boundary between classes of data that can then be used to determine what class new data belongs to. In the context of anomaly detection we have two classes that our data can belong to: normal and anomalous. Where SVMs get particularly interesting is when the classes of data are not linearly separable. In this case, we use something called the kernel trick to linearly separate non-linearly separable data. First, to build up the theory behind SVMs we will start with the simple linearly separable case.

2.1 LINEARLY SEPARABLE DATA

FINDING OUR OPTIMIZATION PROBLEM

When dealing with linearly separable data our goal is to find a hyperplane that correctly separates the data according to its class. Specifically, we want the *optimal* hyperplane so we must start by understanding what makes a hyperplane optimal. An optimal hyperplane classifies the data correctly and has the largest margin between the hyperplane and the points it is classifying.

Take a dataset, $\mathcal{D} = \{(\mathbf{x}_i, y_i) | \mathbf{x}_i \in \mathbb{R}^p, y_i \in \{-1, 1\}\}_{i=1}^n$, with n elements labeled as one of two classes (1 or -1) that are linearly separable. p is an integer representing the dimensionality of the data. We define a hyperplane with a vector $\mathbf{w}$ and a bias b where for each $\mathbf{x} \in \mathcal{D}$, the sign of $\langle \mathbf{w}, \mathbf{x} \rangle + b$ indicates which class $\mathbf{x}$ belongs to. Specifically, $\text{sign}(\langle \mathbf{w}, \mathbf{x} \rangle + b)$ is referred to as the hypothesis function. Until otherwise stated, the standard inner product $\langle -, - \rangle$ refers to the dot product.

We start by defining the **functional margin** of a data point $\mathbf{x}$ with label y as:

$$f = y(\langle \mathbf{w}, \mathbf{x} \rangle + b) \tag{2.1.1}$$

For all points $\mathbf{x} \in \mathcal{D}$ with their labels y , we define the **functional margin on the dataset**:

$$F = \min_{i=1, \dots, n} y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \tag{2.1.2}$$

Here we want to find the hyperplane $(\mathbf{w}, b)$ that has the maximal F value. The important catch here is that we find a loophole: given any hyperplane, one can increase F by scaling up

$\mathbf{w}$ and b . The functional margin is not scale invariant and to rectify this we normalize $\mathbf{w}$ and b . We introduce the **geometric margin**, γ , as well as the **geometric margin on the dataset**, M :

$$\gamma = y \left(\left\langle \frac{\mathbf{w}}{\|\mathbf{w}\|}, \mathbf{x} \right\rangle + \frac{b}{\|\mathbf{w}\|} \right) \quad (2.1.3)$$

$$M = \min_{i=1, \dots, n} y_i \left(\left\langle \frac{\mathbf{w}}{\|\mathbf{w}\|}, \mathbf{x}_i \right\rangle + \frac{b}{\|\mathbf{w}\|} \right) \quad (2.1.4)$$

This formulation of the margin accounts for classification correctness, distance of each point from the hyperplane decision boundary, and is scale invariant. Using the geometric margin, we can now move on to the optimization problem. We want to find the optimal $\mathbf{w}$ and b values to maximize the geometric margin on the dataset, that is:

$$\begin{aligned} & \text{maximize}_{\mathbf{w}, b} \quad M \\ & \text{subject to} \quad \gamma_i \geq M, \quad i = 1, \dots, n \end{aligned} \quad (2.1.5)$$

Note now the relationship between the functional margin and the geometric margin: $M = \frac{F}{\|\mathbf{w}\|}$ and $\gamma = \frac{f}{\|\mathbf{w}\|}$. We will then rewrite (2.1.5) as follows:

$$\begin{aligned} & \text{maximize}_{\mathbf{w}, b} \quad \frac{F}{\|\mathbf{w}\|} \\ & \text{subject to} \quad f_i \geq F, \quad i = 1, \dots, n \end{aligned}$$

Now we can use the fact that the geometric margin is scale invariant and scale $\mathbf{w}$ and b so that $F = 1$ to get:

$$\begin{aligned} & \text{maximize}_{\mathbf{w}, b} \quad \frac{1}{\|\mathbf{w}\|} \\ & \text{subject to} \quad f_i \geq 1, \quad i = 1, \dots, n \end{aligned}$$

By using the definition of the functional margin (2.1.1) and switching to minimization we can easily resolve to:

$$\begin{aligned} & \text{minimize}_{\mathbf{w}, b} \quad \|\mathbf{w}\| \\ & \text{subject to} \quad y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

Finally, we will square the $\|\mathbf{w}\|$ term and a factor of $\frac{1}{2}$ to make optimization easier down the line and avoid the square root. Now, our problem is:

$$\begin{aligned} & \text{minimize}_{\mathbf{w}, b} \quad \frac{1}{2} \|\mathbf{w}\|^2 \\ & \text{subject to} \quad y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1, \quad i = 1, \dots, n \end{aligned} \quad (2.1.6)$$

Now that we have formulated our problem, we look for a method to solve it. $\langle \mathbf{w}, \mathbf{w} \rangle = \|\mathbf{w}\|^2$ is convex and quadratic, our constraints $-y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1$ are linear, and our domain for $\mathbf{w}$ is a subset of $\mathbb{R}^p$, so our problem in (2.1.6) is called a **convex quadratic program**.

First, we will write the *Lagrangian Function* of (2.1.6) with $\alpha_i \geq 0$ as the Lagrange multipliers:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b) - 1] \quad (2.1.7)$$

Here, the Lagrangian primal problem is:

$$\begin{aligned} \min_{\mathbf{w}, b} \max_{\boldsymbol{\alpha}} \quad & \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) \\ \text{subject to} \quad & \alpha_i \geq 0, \quad i = 1, \dots, n \end{aligned} \quad (2.1.8)$$

FINDING THE DUAL REPRESENTATION

We will now find the corresponding dual problem. To start we will differentiate (2.1.7) with respect to $\mathbf{w}$ and b and impose stationarity (that our solution will be a saddle point) which will eventually help us satisfy the necessary the Karush- Kuhn-Tucker (KKT) conditions in the next section:

$$\begin{aligned} \frac{\partial \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = 0 & \implies \mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \\ \frac{\partial \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})}{\partial b} = - \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

Then we then substitute the first relation with $\mathbf{w}$ into (2.1.7):

$$\begin{aligned} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) &= \frac{1}{2} \langle \mathbf{w}, \mathbf{w} \rangle - \sum_{i=1}^n \alpha_i [y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b) - 1] \\ &= \frac{1}{2} \left\langle \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i, \sum_{j=1}^n \alpha_j y_j \mathbf{x}_j \right\rangle - \sum_{i=1}^n \alpha_i \left[y_i \left(\sum_{j=1}^n \alpha_j y_j \langle \mathbf{x}_j, \mathbf{x}_i \rangle + b \right) - 1 \right] \\ &= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle - \sum_{i=1}^n \alpha_i \left[y_i \left(\sum_{j=1}^n \alpha_j y_j \langle \mathbf{x}_j, \mathbf{x}_i \rangle + b \right) - 1 \right] \\ &= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle - \sum_{i=1}^n \alpha_i y_i \sum_{j=1}^n \alpha_j y_j \langle \mathbf{x}_j, \mathbf{x}_i \rangle - b \sum_{i=1}^n \alpha_i y_i + \sum_{i=1}^n \alpha_i \\ &= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle - \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle - b \sum_{i=1}^n \alpha_i y_i + \sum_{i=1}^n \alpha_i \\ &= -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle - b \sum_{i=1}^n \alpha_i y_i + \sum_{i=1}^n \alpha_i \end{aligned}$$

Now we will substitute in the second relation to zero out the term with b and will get what is called a the **Wolfe Dual Lagrangian Function** which we will label $\mathcal{W}(\boldsymbol{\alpha})$:

$$\mathcal{W}(\boldsymbol{\alpha}) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \quad (2.1.9)$$

The corresponding dual problem is:

$$\begin{aligned} \max_{\boldsymbol{\alpha}} \quad & \mathcal{W}(\boldsymbol{\alpha}) \\ \text{subject to} \quad & \alpha_i \geq 0, \quad i = 1, \dots, n \\ & \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned} \quad (2.1.10)$$

For $\boldsymbol{\alpha}^*$ which solves the dual problem, then

$$\mathbf{w}^* = \sum_{i=1}^n \alpha_i^* y_i \mathbf{x}_i \quad (2.1.11)$$

achieves the optimal margin hyperplane according to the geometric margin. Importantly, our target function no longer depends on $\mathbf{w}$ or b which will allow us to do something called the kernel trick cf. Section 2.2. This trick will allow us to work with non-linearly separable data.

THE KARUSH-KUHN-TUCKER CONDITIONS AND SUPPORT VECTORS

Before computing b we need to know what **support vectors** are and their relation to the **Karush-Kuhn-Tucker (KKT) conditions**. The KKT conditions are necessary conditions that the solution to our optimization problem (2.1.10) must satisfy to guarantee an optimal solution and as our problem is convex, the KKT conditions guarantee zero duality gap [7]. The KKT conditions must be imposed because our optimization problem (2.1.10) has inequality conditions. I referenced [6] for their presentation of the KKT conditions. The conditions and how we satisfy them are as follows:

Stationarity condition: The KKT conditions require that our solution be a saddle point of $\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})$ which we imposed by setting $\frac{\partial \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})}{\partial \mathbf{w}} = 0$ and $\frac{\partial \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})}{\partial b} = 0$.

Primal feasibility condition: Our solution must satisfy the original constraints of the primal problem, that is $y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1, \quad i = 1, \dots, n$ from (2.1.6).

Dual feasibility condition: Our solution must satisfy the constraints of the dual problem, that is $\alpha_i \geq 0, \quad i = 1, \dots, n$ from (2.1.10).

Complementary slackness condition: Our solution must satisfy the following:
 $\alpha_i[y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) - 1] = 0, \quad i = 1, \dots, n$.

The complementary slackness condition implies that the only inputs $\mathbf{x}_i$ for which α_i can be non-zero are those in which the functional margin (2.1.1) is 1. Thus, in the expression of the weight vector $\mathbf{w}^* = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$ from (2.1.11) the only inputs that are not zeroed out are the inputs with a functional margin of 1. These are the **support vectors** – shown in Figure 1. The support vectors are the only data points that contribute to the decision boundary – they *support* the hyperplane itself.

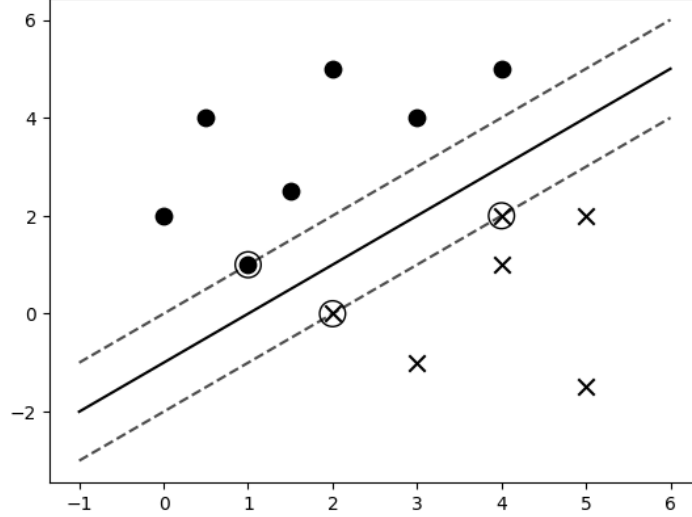


Figure 1: An example support vector machine with support vectors circled.

SOLVING FOR b

As the dual problem (2.1.10) does not provide information about b we must use the constraints of the original optimization problem (2.1.6) to find the solution b^* : $y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1$

We multiply both sides by y_i as $y_i^2 = 1$ and reformat to find: $b = y_i - \langle \mathbf{w}, \mathbf{x}_i \rangle$

There are multiple ways to go about calculating b^* – the first we will discuss is from [6]. Now, technically we can choose any support vector, $\mathbf{x}_i$, to solve for b but we prefer a consistent solution. Let S be the set of support vectors. We use the average of the b values produced by the support vectors:

$$b^* = \frac{1}{\#S} \sum_{i=1}^{\#S} (y_i - \langle \mathbf{w}, \mathbf{x}_i \rangle) \quad (2.1.12)$$

The second method to calculate b^* was the original formula used when Vapnik and Cortes introduced the support vector machine [1] and was used in [2]. This method uses the average of the closest examples in each class to the hyperplane. That is:

$$b^* = -\frac{\max_{y_i=-1}(\langle \mathbf{w}^*, \mathbf{x}_i \rangle) + \min_{y_i=1}(\langle \mathbf{w}^*, \mathbf{x}_i \rangle)}{2} \quad (2.1.13)$$

Briefly, we can see how (2.1.13) follows from the original conditions:

$$\begin{aligned} & \frac{\max_{y_i=-1}(y_i - \langle \mathbf{w}^*, \mathbf{x}_i \rangle) + \min_{y_i=1}(y_i - \langle \mathbf{w}^*, \mathbf{x}_i \rangle)}{2} \\ & \frac{\max_{y_i=-1}(-1 - \langle \mathbf{w}^*, \mathbf{x}_i \rangle) + \min_{y_i=1}(1 - \langle \mathbf{w}^*, \mathbf{x}_i \rangle)}{2} \\ & \frac{-1 - \max_{y_i=-1}(\langle \mathbf{w}^*, \mathbf{x}_i \rangle) + 1 - \min_{y_i=1}(\langle \mathbf{w}^*, \mathbf{x}_i \rangle)}{2} \\ & - \frac{\max_{y_i=-1}(\langle \mathbf{w}^*, \mathbf{x}_i \rangle) + \min_{y_i=1}(\langle \mathbf{w}^*, \mathbf{x}_i \rangle)}{2} \end{aligned}$$

2.2 NON-LINEAR CLASSIFICATION

Unfortunately, data is usually not nicely linearly separable so we have to modify our approach to finding the decision boundary. One situation in which nonlinearly separable data can arise is in datasets that *should* display linearly separable classes but have noisy data. This noisy data can be the result of anything from a faulty sensor reporting incorrect readings to a typo in the dataset recorded by a researcher.

Noise in the dataset can result in hyperplanes with extremely small margins and misclassified results. In Figure 2 we can see that the introduction of an outlier results in a hyperplane with an extremely low margin (on the right). This classifier will generalize much poorer than the one on the left when deployed because of this shift in the decision boundary. If the outlier point broke the linear separability of the classes, our Hard Margin Classifier would be incapable of determining a final hyperplane.

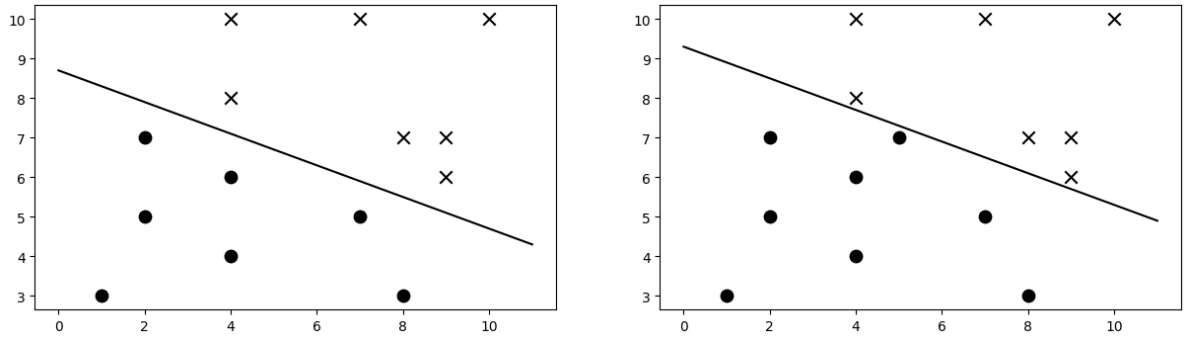


Figure 2: Hard Margin Classifiers for the same dataset without an outlier, left, and with an outlier at (5, 7), right. Example dataset from [6].

To deal with potential noise in our data blurring the lines between linearly separable classes we will define the **Soft Margin SVM**. We can no longer guarantee an “optimal” hyperplane as we defined it before - that is, one hyperplane may not be able to correctly classifying every point at the same time and, moreover, the “best” hyperplane may actually be one that misclassifies a point. The method outlined in Section 2.1 is referred to as a **Maximal Margin Classifier** and we must now define the Soft Margin SVM.

Our guiding idea with the Soft Margin SVM is that we want to allow for potential misclassifications in the training dataset (within reason). Recall our primal optimization problem before introducing the Lagrange multipliers (2.1.6):

$$\begin{aligned} & \text{minimize}_{\mathbf{w}, b} \quad \frac{1}{2} \|\mathbf{w}\|^2 \\ & \text{subject to} \quad y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

We will introduce slack variables to the constraints of this problem to allow for violation of the original constraints:

$$\begin{aligned} & \text{subject to} \quad y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i, \quad i = 1, \dots, n \\ & \quad \quad \quad \xi_i \geq 0, \quad i = 1, \dots, n \end{aligned}$$

Now, to avoid the selection of arbitrary large values for ξ_i which would allow us to easily satisfy all constraints we modify the equation as follows:

$$\begin{aligned} & \text{minimize}_{\mathbf{w}, b} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ & \text{subject to} \quad y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i, \quad i = 1, \dots, n \\ & \quad \quad \quad \xi_i \geq 0, \quad i = 1, \dots, n \end{aligned} \tag{2.2.1}$$

We added the parameter C here so we can control how “soft” our margin is. Adjusting C depending on our data and the type of noise allows us to find the best hyperplane that balances the number of training points that are misclassified or close to the hyperplane allowed. This will allow us to reduce the impact of noisy data on the final solution.

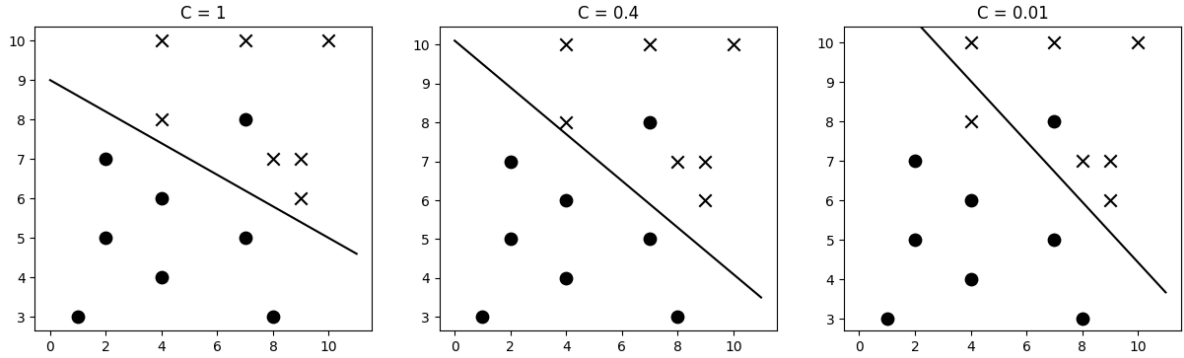


Figure 3: The same dataset with one outlier at (7, 8) with C values of 1, 0.4, and 0.01 as noted on the plots.

As shown in Figure 3, generally speaking smaller values of C allow for larger values of ξ_i which will result in more misclassifications. Larger C values prioritize smaller ξ_i values and thus less misclassifications.

The formulation of our soft margin primal optimization problem (2.2.1) is referred to as the **1-norm soft margin** because we are working with the 1-norm of the vector ξ , our slack variable.

Now we will find the Wolfe dual formulation of the problem, starting with the corresponding Lagrangian:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=0}^n \xi_i - \sum_{i=0}^n \alpha_i [y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b) - 1 + \xi_i] - \sum_{i=0}^n \mu_i \xi_i \quad (2.2.2)$$

As before, we will impose stationarity (one of our KKT Conditions) by differentiating with respect to $\mathbf{w}$, b , and $\boldsymbol{\xi}$, and setting the derivatives to zero:

$$\frac{\partial \mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu})}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=0}^n \alpha_i y_i \mathbf{x}_i = 0 \implies \mathbf{w} = \sum_{i=0}^n \alpha_i y_i \mathbf{x}_i$$

$$\frac{\partial \mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu})}{\partial b} = \sum_{i=0}^n \alpha_i y_i = 0$$

$$\frac{\partial \mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu})}{\partial \xi_i} = C - \alpha_i - \mu_i = 0$$

Now, before we substitute these new identities into the Lagrangian (2.2.2), let's rearrange it first:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=0}^n \alpha_i [y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b) - 1] - \sum_{i=0}^n \alpha_i \xi_i + C \sum_{i=0}^n \xi_i - \sum_{i=0}^n \mu_i \xi_i$$

The first two terms in this form are exactly the Lagrangian of our hard margin classifier (2.1.7) so let's look at the last three terms:

$$\begin{aligned} & - \sum_{i=0}^n \alpha_i \xi_i + C \sum_{i=0}^n \xi_i - \sum_{i=0}^n \mu_i \xi_i \\ & \sum_{i=0}^n C \xi_i - \sum_{i=0}^n \alpha_i \xi_i - \sum_{i=0}^n \mu_i \xi_i \\ & \sum_{i=0}^n (C \xi_i - \alpha_i \xi_i - \mu_i \xi_i) \\ & \sum_{i=0}^n \xi_i (C - \alpha_i - \mu_i) \end{aligned}$$

And we recently found $C - \alpha_i - \mu_i = 0$ we we now know

$$- \sum_{i=0}^n \alpha_i \xi_i + C \sum_{i=0}^n \xi_i - \sum_{i=0}^n \mu_i \xi_i = 0$$

With only the first two terms left in our original Lagrangian (2.2.2) we find that the Wolfe dual form and the dual optimization problem of the soft margin SVM is the same as the hard margin classifier but with a slightly different constraint. Our Wolfe dual form:

$$\mathcal{W}(\boldsymbol{\alpha}) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle$$

With the dual optimization problem:

$$\begin{aligned} \max_{\boldsymbol{\alpha}} \quad & \mathcal{W}(\boldsymbol{\alpha}) \\ \text{subject to} \quad & 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n \\ & \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned} \tag{2.2.3}$$

The condition $0 \leq \alpha_i \leq C$ arises because $C - \alpha_i - \mu_i = 0$ and $\mu_i \geq 0$ as it is one of our Lagrange multipliers and we have inequality constraints. This constraint is referred to as the **box constraint** because $\boldsymbol{\alpha}$ must lie within the box with side length C in the positive orthant.

Much like before, for $\boldsymbol{\alpha}^*$ which solves the dual optimization problem, $\mathbf{w}^* = \sum_{i=1}^n y_i \alpha_i \mathbf{x}_i$. Before we can choose b^* we need to understand what the support vectors are for a soft margin SVM so we return once more to the KKT conditions. This time, our complementary slackness condition implies that:

$$\begin{aligned} \alpha_i [y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b) - 1 + \xi_i] &= 0 & i = 1, \dots, n \\ \xi_i (\alpha_i - C) &= 0 & i = 1, \dots, n \end{aligned}$$

These conditions mean that non-zero slack variables only happen when $\alpha_i = C$. The points that have non-zero slack variables are within the margin of $\frac{1}{\|\mathbf{w}\|}$ of the hyperplane – earlier this was the margin of 1 as we had normalized $\mathbf{w}$ but the function is the same here. When $\alpha_i = 0$ our point has been correctly classified and is outside the $\frac{1}{\|\mathbf{w}\|}$ margin. That just leaves points for which we have the strict inequality $0 < \alpha_i < C$. These points lie exactly on the $\frac{1}{\|\mathbf{w}\|}$ margin and are our support vectors. Now that we have our support vectors we can use either method of finding b^* : (2.1.12) or (2.1.13).

THE KERNEL TRICK

We have seen how to deal with nonlinearities in data due to noise. These nonlinearities arise specifically when the data *should* be in linearly separable classes but it is not due to a certain “blurring” of the edges. The soft margin SVM helped us overcome this challenge but now we face the next problem: data that has classes which are inherently nonlinearly separable in their original dimensionality.

Take, for example the dataset in Figure 4. This dataset is not linearly separable in 2 dimensions but if we were to map the dataset into a 3 dimensional space using a feature map $\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^3$

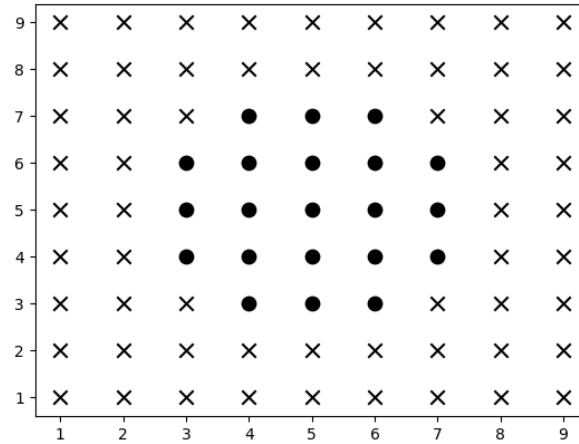


Figure 4: A nonlinearly separable dataset.

defined as $\phi(x, y) = (x, y, (x - 5)^2 + (y - 5)^2)$, suddenly things become much more linearly separable! This map can be seen in Figure 5.

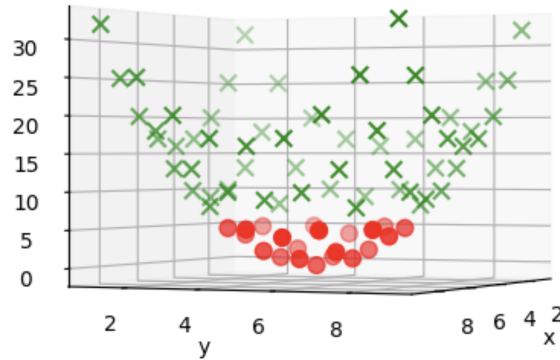


Figure 5: The same dataset as Figure 4 mapped into 3 dimensional space.

So, if we want to linearly separate our difficult datasets, we could transform our data into a higher-dimensional *feature space* where the data would be linearly separable and train our SVM on that data and then transform any data we wanted to run class prediction on. Unfortunately, transforming every single data point we use is a lot of work – especially if our transformation is complicated – and will hugely slow down the training time and inference speed of our SVM. This is where we introduce the kernel trick, a very important hallmark of the SVM.

First we will revisit the final formulation of our problem that we found with the Wolfe dual

representation of the soft margin SVM:

$$\begin{aligned} \max_{\boldsymbol{\alpha}} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \\ \text{subject to} \quad & 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n \\ & \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

We make one very important observation here – we do not actually need to know the individual values of any one datapoint $\mathbf{x}_i$, we only need to know the value of the dot product, $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$.

Example: Let us return to our example illustrated in Figure 4 and Figure 5. One way to make our data linearly separable is to map it all using the feature map we defined as $\phi(a, b) = (a, b, (a - 5)^2 + (b - 5)^2)$. Our Wolfe dual formulation of the soft margin SVM is then concerned with:

$$\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle = x_1 y_1 + x_2 y_2 + [(x_1 - 5)^2 + (x_2 - 5)^2][(y_1 - 5)^2 + (y_2 - 5)^2]$$

Importantly, we realize that instead of transforming each datapoint we can use this formula directly. We will name it, $K(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle$ and adjust our optimization problem:

$$\begin{aligned} \max_{\boldsymbol{\alpha}} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \\ \text{subject to} \quad & 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n \\ & \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

K is the eponymous kernel function of the kernel trick. With this formulation we no longer have to transform all of our data to reap the benefits that a higher dimensional feature space can provide for us.

We more formally define a **kernel function**: Let $\mathcal{X}$ be our input space, typically $\mathbb{R}^p$ where p is the dimension of the data. Given a mapping function $\phi : \mathcal{X} \rightarrow \mathcal{V}$, the function $K : \mathcal{X} \rightarrow \mathbb{R}$ defined as $K(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle_{\mathcal{V}}$ where $\langle -, - \rangle_{\mathcal{V}}$, denotes an inner product in the feature space $\mathcal{V}$, is called a kernel function.

From this definition we can easily see that $K(\mathbf{x}, \mathbf{y}) = \langle x, y \rangle$ is a satisfactory kernel that we have been using until now. The important catch with this definition is that it is not always easy to define the feature space and its inner product or the feature map explicitly so we look for a way to define a kernel without first defining our feature map and subsequent feature space. To find a kernel function that implicitly defines our feature space we have some requirements laid out by the following theorem.

Theorem: Let $\mathcal{X}$ be a finite input space with symmetric function $K(\mathbf{x}, \mathbf{y})$ on $\mathcal{X}$. Then $K(\mathbf{x}, \mathbf{y})$ is a kernel function if and only if the Gramian matrix

$$\mathbf{K} = (K(\mathbf{x}_i, \mathbf{x}_j))_{i,j=1}^n$$

has only non-negative eigenvalues (is positive semi-definite).

To briefly show why this holds, consider the eigenvalues, λ_i , and eigenvectors, $\mathbf{v}_i = (v_{ij})_{j=1}^n$, of K . We define the feature mapping $\phi(\mathbf{x}_i) = \sqrt{\lambda_i} \mathbf{v}_i$. We can use this feature mapping to implicitly define K and our feature space:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle = \sum_{k=1}^n \lambda_k v_{ik} v_{jk}$$

Say we have a negative eigenvalue, λ_l , with eigenvector $\mathbf{v}_l$ then the point defined by

$$\mathbf{w} = \sum_{i=0}^n v_{li} \phi(\mathbf{x}_i) = \sum_{i=0}^n v_{li} \sqrt{\lambda_i} \mathbf{v}_i = \mathbf{v}_l \sum_{i=0}^n \sqrt{\lambda_i} \mathbf{v}_i$$

We can see that $\mathbf{w}$ can be represented by $\mathbf{v}_l$, the diagonal matrix Λ of eigenvalues, and V^{-1} the transposed matrix of eigenvectors of K . That is:

$$\mathbf{w} = \sqrt{\Lambda} V^{-1} \mathbf{v}_l$$

We see that $\mathbf{w}$ will have norm squared:

$$\|\mathbf{w}\|^2 = \langle \mathbf{w}, \mathbf{w} \rangle = \mathbf{v}_l^{-1} V \Lambda V^{-1} \mathbf{v}_l = \mathbf{v}_l^{-1} K \mathbf{v}_l = \lambda_l$$

From this, we can see that a negative eigenvalue would produce a negative value for the norm squared of a vector in the feature space which is a contradiction.

This is a corollary of **Mercer's Theorem** (cf. [Appendix A](#)) which deals more broadly with compact subsets of $\mathbb{R}^n$.

In this new kernelized formulation of our SVM, we have a slightly different decision function:

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$

This prevents us from needing to explicitly calculate $\mathbf{w}$ and arises from the KKT condition requirements. The class of our data will be determined by the sign of this function.

COMMON TYPES OF KERNELS

Linear Kernel: Arguably the simplest kernel type is the linear kernel defined as $K(\mathbf{x}, \mathbf{y}) = \mathbf{x} \cdot \mathbf{y}$. This is just the dot product formulation we have been using until now. Linear kernels are commonly used for extremely high-dimensional data due to their relatively simple calculation.

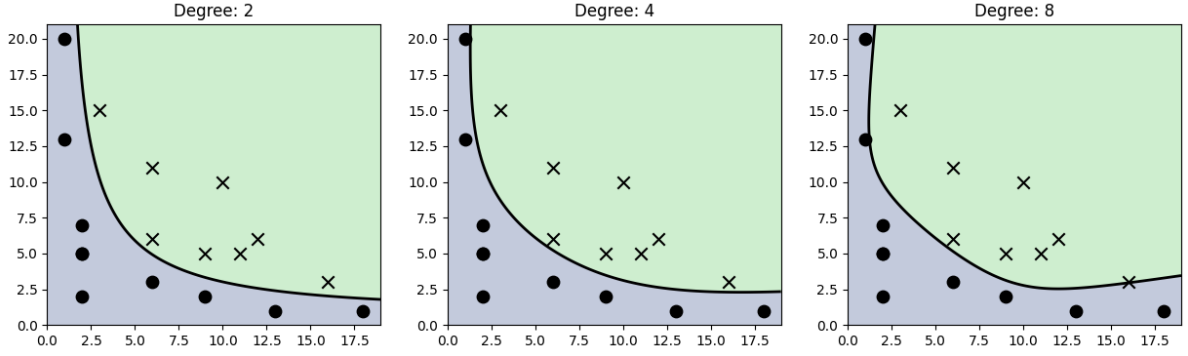


Figure 6: SVM with polynomial kernels of varying degree on the same dataset.

Polynomial Kernel: With the polynomial kernel we introduce some variables: $d \in \mathbb{Z}^+$ which corresponds to the desired degree of our polynomial and $c \in \mathbb{R}, c \geq 0$ as our constant term. The kernel is defined as $K(\mathbf{x}, \mathbf{y}) = (\mathbf{x} \cdot \mathbf{y} + c)^d$.

When increasing the degree of our polynomial kernel, the decision boundary becomes more complex and can become overfitted. An illustration of how polynomial kernel degree variation can impact the decision boundary can be seen in Figure 6.

Radial Basis Function (RBF) or Gaussian Kernel: This is the most commonly used kernel and often the most powerful kernel. The RBF kernel is defined as $K(\mathbf{x}, \mathbf{y}) = \exp(-\gamma \|\mathbf{x} - \mathbf{y}\|^2)$ where $\gamma \in \mathbb{R}^+$. This kernel is only concerned with the distance between the two inputs. As γ approaches zero, a SVM with a RBF kernel will behave as a linear classifier – alternatively, when γ is too large it can lead to overfitting. Figure 7 shows an example of the impact of γ value variation.

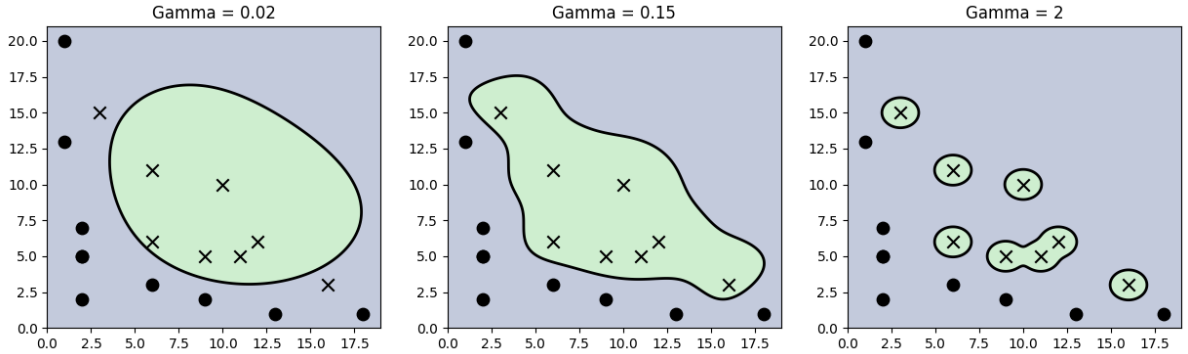


Figure 7: SVM with RBF kernels with varying values of γ .

You may see the decision boundaries produced by the RBF kernel in Figure 7 and think back to our example of a nonlinearly separable dataset in Figure 4 and Figure 5. Indeed, the RBF kernel can be used to effectively classify this data as seen in Figure 8.

These are only a few examples of kernels though there are many different options. In general, a kernel is trying to estimate how similar two vectors are so domain knowledge about one's

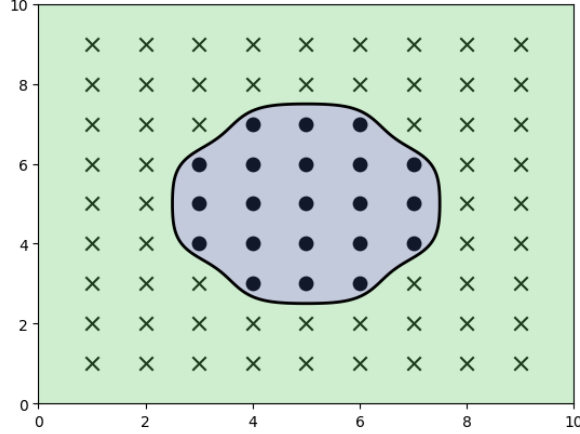


Figure 8: SVM with RBF kernel, $\gamma = 1$, on the dataset from Figure 4.

particular dataset could help in the search for the right kernel.

2.3 SEQUENTIAL MINIMAL OPTIMIZATION (SMO) ALGORITHM

We have now spent considerable effort formulating our final optimization problem but we have not actually come up with a method to solve it yet. Generally, when solving convex optimization problems one would use optimization libraries like CVXOPT for Python and other quadratic programming solvers. These methods, though effective, are quite slow and involve large matrix operations which not only take a long time but massive amounts of memory. We want something a little more tailored to our problem. We find our answer with the **Sequential Minimal Optimization (SMO) Algorithm**.

The SMO algorithm was designed specifically for solving the SVM optimization problem by John Platt in 1998 [11]. The core of the SMO algorithm is the idea that instead of trying to optimize every term of α at the same time using an iterative QP solver, we can instead choose two, α_a and α_b , $a \neq b$, to optimize at a given time. Due to the condition $\sum_{i=0}^n \alpha_i y_i = 0$ we must optimize at least two terms of α at once so that we can guarantee the sum still holds at zero. At each step we will use heuristics to choose which two terms of α we will optimize. We will find an analytic solution to the optimization problem involving just two terms of α .

Though this method requires more steps than an algorithm like gradient descent or a QP solver, each individual step is so much faster due to the analytic solution that requires no matrix operations that it is much faster overall.

ANALYTICAL SOLUTION FOR TWO LAGRANGE MULTIPLIERS

Without loss of generality we will choose two Lagrange multipliers α_a and α_b to optimize. The final optimized versions of α_a and α_b will be denoted α'_a and α'_b . In order to adhere to the

condition $\sum_{i=0}^n \alpha_i y_i = 0$ we know we must have $\alpha_a y_a + \alpha_b y_b = \alpha'_a y_a + \alpha'_b y_b = l$, where l is some constant. We also know that we must satisfy $0 \leq \alpha_a, \alpha_b, \alpha'_a, \alpha'_b \leq C$ as it is one of our constraints. These two conditions together mean our Lagrange multipliers must lie on a diagonal line within a square of side length C .

We will first compute α_b and then use it to find α_a . Let us first consider the case in which $y_a = y_b$ and without loss of generality we choose the positive class, $y_a = y_b = 1$. Then we know $\alpha_a + \alpha_b = \alpha'_a + \alpha'_b$. We want to examine the inequality $0 \leq \alpha'_b \leq C$ which we know holds for α'_a as well.

We will use f to denote the hypothesis function determined by the current values of α . That is:

$$f(\mathbf{z}) = K(\mathbf{w}, \mathbf{z}) + b = \sum_{i=1}^n \alpha_i y_i K(\mathbf{x}_i, \mathbf{z}) + b$$

Consider $\alpha'_a = C$, then we have $\alpha_a + \alpha_b = C + \alpha'_b \implies \alpha'_b = \alpha_a + \alpha_b - C$. Similarly with $\alpha'_a = 0$ we find $\alpha'_b = \alpha_a + \alpha_b$. We can use this information to construct a slightly more restrictive inequality around α'_b :

$$\max(0, \alpha_a + \alpha_b - C) \leq \alpha'_b \leq \min(C, \alpha_a + \alpha_b)$$

We can analogously compute the case in which $y_a \neq y_b$ to find:

$$\max(0, \alpha_a - \alpha_b) \leq \alpha'_b \leq \min(C, C - \alpha_a + \alpha_b)$$

We will name these bounds:

$$U = \begin{cases} \max(0, \alpha_a + \alpha_b - C), & \text{if } y_a = y_b \\ \max(0, \alpha_a - \alpha_b), & \text{if } y_a \neq y_b \end{cases} \quad (2.3.1)$$

$$V = \begin{cases} \min(C, \alpha_a + \alpha_b), & \text{if } y_a = y_b \\ \min(C, C - \alpha_a + \alpha_b), & \text{if } y_a \neq y_b \end{cases} \quad (2.3.2)$$

We will now consider our objective function:

$$\mathcal{W}(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

To simplify notation we write $K(\mathbf{x}_i, \mathbf{x}_j) = K_{ij}$. We have decided to only manipulate α_a and α_b so we will look at $\mathcal{W}$ as a function of just α_a and α_b :

$$\begin{aligned} \mathcal{W}(\alpha_a, \alpha_b) = & \alpha_a + \alpha_b - \frac{1}{2} \alpha_a^2 K_{aa} - \frac{1}{2} \alpha_b K_{bb} - \alpha_a \alpha_b y_a y_b K_{ab} \\ & - \alpha_a y_a \sum_{i=1, i \neq a, b}^n \alpha_i y_i K_{ai} - \alpha_b y_b \sum_{i=1, i \neq a, b}^n \alpha_i y_i K_{bi} + \text{constant} \end{aligned}$$

We will also define

$$v_a = \sum_{i=1, i \neq a, b}^n \alpha_i y_i K_{ai} = f(\mathbf{x}_a) - y_a \alpha_a K_{aa} - y_b \alpha_b K_{ab} - b$$

$$v_b = \sum_{i=1, i \neq a, b}^n \alpha_i y_i K_{bi} = f(\mathbf{x}_b) - y_a \alpha_a K_{ba} - y_b \alpha_b K_{bb} - b$$

so we can simplify further:

$$\mathcal{W}(\alpha_a, \alpha_b) = \alpha_a + \alpha_b - \frac{1}{2} \alpha_a^2 K_{aa} - \frac{1}{2} \alpha_b^2 K_{bb} - \alpha_a \alpha_b y_a y_b K_{ab} - \alpha_a y_a v_a - \alpha_b y_b v_b + \text{constant}$$

Now, as we know from our imposed conditions $y_a \alpha_a + y_b \alpha_b = y_a \alpha'_a + y_b \alpha'_b$ is a constant and we will call it γ . We will also label $y_a y_b = s$ to get $\alpha_a + s \alpha_b = \gamma$. We can now use this equation to substitute $\alpha_a = \gamma - s \alpha_b$ in our equation:

$$\begin{aligned} \mathcal{W}(\alpha_b) &= \gamma - s \alpha_b + \alpha_b - \frac{1}{2} (\gamma - s \alpha_b)^2 K_{aa} - \frac{1}{2} \alpha_b^2 K_{bb} - (\gamma - s \alpha_b) \alpha_b s K_{ab} \\ &\quad - (\gamma - s \alpha_b) y_a v_a - \alpha_b y_b v_b + \text{constant} \end{aligned}$$

From here we will bring back our stationary condition:

$$\frac{\partial \mathcal{W}(\alpha_b)}{\partial \alpha_b} = -s + 1 + s K_{aa} (\gamma - s \alpha_b) - K_{bb} \alpha_b + K_{ab} \alpha_b - s K_{ab} (\gamma - s \alpha_b) + y_b v_a - y_b v_b = 0$$

From here we can try to solve for α_b^* :

$$\begin{aligned} \alpha_b^* (K_{aa} + K_{bb} - 2K_{ab}) &= -s + 1 + s K_{aa} \gamma - s K_{ab} \gamma + y_b v_a - y_b v_b \\ &= 1 - s + s \gamma (K_{aa} - K_{ab}) + y_b (v_a - v_b) \end{aligned}$$

We can see that:

$$\begin{aligned} v_a - v_b &= f(\mathbf{x}_a) - y_a \alpha_a K_{aa} - y_b \alpha_b K_{ab} - b - f(\mathbf{x}_b) + y_a \alpha_a K_{ba} + y_b \alpha_b K_{bb} + b \\ &= f(\mathbf{x}_a) - f(\mathbf{x}_b) + y_a \alpha_a (K_{ba} - K_{aa}) + y_b \alpha_b (K_{bb} - K_{ab}) \end{aligned}$$

We will substitute this back in for $v_a - v_b$ and multiply both sides by y_b :

$$\begin{aligned} \alpha_b^* y_b (K_{aa} + K_{bb} - 2K_{ab}) &= y_b - y_a + y_a \gamma (K_{aa} - K_{ab}) \\ &\quad + f(\mathbf{x}_a) - f(\mathbf{x}_b) + y_a \alpha_a (K_{ba} - K_{aa}) + y_b \alpha_b (K_{bb} - K_{ab}) \end{aligned}$$

We will now substitute back in $\alpha_a + s\alpha_b = \gamma$:

$$\begin{aligned}
\alpha_b^* y_b (K_{aa} + K_{bb} - 2K_{ab}) &= y_b - y_a + f(\mathbf{x}_a) - f(\mathbf{x}_b) + y_a(\alpha_a + s\alpha_b)(K_{aa} - K_{ab}) \\
&\quad + y_a\alpha_a(K_{ba} - K_{aa}) + y_b\alpha_b(K_{bb} - K_{ab}) \\
&= y_b - y_a + f(\mathbf{x}_a) - f(\mathbf{x}_b) + y_a\alpha_a K_{aa} - y_a\alpha_a K_{ab} + y_b\alpha_b K_{aa} \\
&\quad - y_b\alpha_b K_{ab} + y_a\alpha_a K_{ba} - y_a\alpha_a K_{aa} + y_b\alpha_b K_{bb} - y_b\alpha_b K_{ab} \\
&= y_b - y_a + f(\mathbf{x}_a) - f(\mathbf{x}_b) + y_b\alpha_b(K_{aa} - 2K_{ab} + K_{bb})
\end{aligned}$$

So with a little rearranging:

$$\alpha_b^* = \alpha_b + \frac{y_b[(f(\mathbf{x}_a) - y_a) - (f(\mathbf{x}_b) - y_b)]}{K_{aa} + K_{bb} - 2K_{ab}} \quad (2.3.3)$$

Now the only thing we have left to do is enforce the inequalities we found in eq. (2.3.1) and eq. (2.3.2). We refer to this as clipping the value of α_b' :

$$\alpha_b' = \begin{cases} V, & \text{if } \alpha_b^* > V \\ \alpha_b^*, & \text{if } U \leq \alpha_b^* \leq V \\ U, & \text{if } \alpha_b^* < U \end{cases} \quad (2.3.4)$$

And just like that we have our update rule for the SMO algorithm! We will update α_b first and then use $y_a\alpha_a + y_b\alpha_b = y_a\alpha_a' + y_b\alpha_b'$ to compute α_a' .

HEURISTICS TO CHOOSE α_a AND α_b

Now that we know how we will be updating our chosen Lagrange multipliers, we need to know exactly how we will be choosing them.

Choosing α_b : We will start by choosing a point $\mathbf{x}_b$ that violates the KKT conditions as they are necessary for a solution. The main loop of the SMO algorithm will loop through the training set looking specifically at points with α_b values that satisfy the strict inequality $0 < \alpha_b < C$ which increases our chances of finding a point that violates the KKT conditions. Only once all points that satisfy $0 < \alpha_b < C$ are in accordance with the KKT conditions will we look at the rest of the points.

Choosing α_a : We want to choose the point $\mathbf{x}_a$ such that we have a large change when updating α_a and α_b as this will result in a correspondingly large change in the dual objective. We will look at $|(f(\mathbf{x}_a) - y_a) - (f(\mathbf{x}_b) - y_b)|$ which should look familiar as it appears in (2.3.3). These quantities are sometimes labeled as follows: $|E_a - E_b|$ and if E_a is positive we will maximize E_b whereas if E_a is negative we will minimize E_b .

There is some more nuance to the SMO algorithm in terms of how memory is saved and how computation is sped up. I have included my implementation of this algorithm in Python in cf. [Appendix B](#).

3 VARIATIONS OF THE SUPPORT VECTOR MACHINE

3.1 ONE-CLASS SUPPORT VECTOR MACHINES

One-Class Support Vector Machines (OC-SVMs) are an important variant of SVMs as they allow us to do unsupervised learning rather than the supervised learning of traditional SVMs. Supervised learning refers to machine learning algorithms that take in data with labels (in our case, the y_i values are labels) whereas in unsupervised learning, the machine learning algorithms take in unlabeled data. In real world scenarios, labeled data can be hard to acquire – especially as data typically requires hand-labeling or at least hand-checking of the labels resulting in a huge cost in human time.

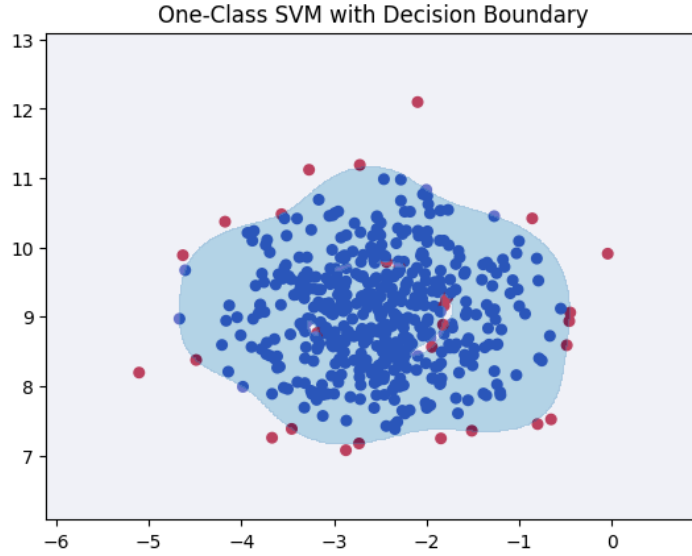


Figure 9: A OC-SVM on a Gaussian blob using the Radial Basis Function as a kernel and $\nu = 0.05$

Introduced in [13], OC-SVMs take in data that belongs to one class (hence the name) and instead of finding a separating hyperplane, they look to find an enclosing boundary for the given training data. The OC-SVM then uses this enclosing boundary to determine if new data is “normal”/typically aligns with the given training data, or if it is anomalous and represents an outlier. This makes OC-SVMs very good at anomaly detection tasks like monitoring WSNs.

The changes in the primal optimization problem for OC-SVMs are minor with the introduction of ν which we will use as an analogue of C to control the proportion of errors allowed in classification of our training dataset and ρ which will act like our b value previously did. Assume

our kernel function implicitly defines $K(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle$. The equation is as follows:

$$\begin{aligned} \min_{\mathbf{w}, \boldsymbol{\xi}, \rho} \quad & \frac{1}{2} \|\mathbf{w}\|^2 - \rho + \frac{1}{\nu n} \sum_{i=0}^n \xi_i \\ \text{subject to} \quad & \langle \mathbf{w}, \phi(\mathbf{x}_i) \rangle \geq \rho - \xi_i, \quad i = 1, \dots, n \\ & \xi_i \geq 0, \quad i = 1, \dots, n \end{aligned} \quad (3.1.1)$$

Our dual problem can be derived in the same way we found it for the standard SVM. To begin we write the primal Lagrangian:

$$\mathcal{L}(\mathbf{w}, \rho, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \frac{1}{2} \|\mathbf{w}\|^2 - \rho + \frac{1}{\nu n} \sum_{i=0}^n \xi_i - \sum_{i=0}^n \alpha_i [\langle \mathbf{w}, \phi(\mathbf{x}_i) \rangle - \rho + \xi_i] - \sum_{i=0}^n \mu_i \xi_i \quad (3.1.2)$$

We first impose stationarity by differentiating with respect to $\mathbf{w}$, ρ , and $\boldsymbol{\xi}$, and setting the derivatives to zero:

$$\begin{aligned} \frac{\partial \mathcal{L}(\mathbf{w}, \rho, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu})}{\partial \mathbf{w}} &= \mathbf{w} - \sum_{i=0}^n \alpha_i \phi(\mathbf{x}_i) = 0 \implies \mathbf{w} = \sum_{i=0}^n \alpha_i \phi(\mathbf{x}_i) \\ \frac{\partial \mathcal{L}(\mathbf{w}, \rho, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu})}{\partial \rho} &= \sum_{i=0}^n \alpha_i - 1 = 0 \implies 1 = \sum_{i=0}^n \alpha_i \\ \frac{\partial \mathcal{L}(\mathbf{w}, \rho, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu})}{\partial \xi_i} &= \frac{1}{\nu n} - \alpha_i - \mu_i = 0 \end{aligned}$$

Before we substitute these new identities into the Lagrangian (3.1.2) we rearrange it:

$$\mathcal{L}(\mathbf{w}, \rho, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \frac{1}{2} \|\mathbf{w}\|^2 - \rho + \frac{1}{\nu n} \sum_{i=0}^n \xi_i - \sum_{i=0}^n \alpha_i \langle \mathbf{w}, \phi(\mathbf{x}_i) \rangle + \rho \sum_{i=0}^n \alpha_i - \sum_{i=0}^n \alpha_i \xi_i - \sum_{i=0}^n \mu_i \xi_i$$

Gathering the terms with ξ_i we see:

$$\begin{aligned} & \frac{1}{\nu n} \sum_{i=0}^n \xi_i - \sum_{i=0}^n \alpha_i \xi_i - \sum_{i=0}^n \mu_i \xi_i \\ & \sum_{i=0}^n \xi_i \left(\frac{1}{\nu n} - \alpha_i - \mu_i \right) = \sum_{i=0}^n \xi_i (0) = 0 \end{aligned}$$

Now our Lagrangian simplifies:

$$\mathcal{L}(\mathbf{w}, \rho, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \rho - \sum_{i=0}^n \alpha_i \langle \mathbf{w}, \phi(\mathbf{x}_i) \rangle + \rho \sum_{i=0}^n \alpha_i$$

As we have shown $\sum_{i=0}^n \alpha_i = 1$ the terms with ρ also cancel:

$$\mathcal{L}(\mathbf{w}, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=0}^n \alpha_i \langle \mathbf{w}, \phi(\mathbf{x}_i) \rangle$$

Now we substitute in our identity $\mathbf{w} = \sum_{i=0}^n \alpha_i \phi(\mathbf{x}_i)$:

$$\begin{aligned} \mathcal{L}(\boldsymbol{\alpha}) &= \frac{1}{2} \left\langle \sum_{i=0}^n \alpha_i \phi(\mathbf{x}_i), \sum_{i=0}^n \alpha_i \phi(\mathbf{x}_i) \right\rangle - \sum_{i=0}^n \alpha_i \left\langle \sum_{j=0}^n \alpha_j \phi(\mathbf{x}_j), \phi(\mathbf{x}_i) \right\rangle \\ \mathcal{L}(\boldsymbol{\alpha}) &= \frac{1}{2} \sum_{i=0}^n \sum_{j=0}^n \alpha_i \alpha_j \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle - \sum_{i=0}^n \sum_{j=0}^n \alpha_i \alpha_j \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_i) \rangle \\ \mathcal{L}(\boldsymbol{\alpha}) &= -\frac{1}{2} \sum_{i=0}^n \sum_{j=0}^n \alpha_i \alpha_j \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle \\ \mathcal{L}(\boldsymbol{\alpha}) &= -\frac{1}{2} \sum_{i=0}^n \sum_{j=0}^n \alpha_i \alpha_j K(\mathbf{x}_i, \mathbf{x}_j) \end{aligned}$$

We have now recovered our Wolfe Dual form of the OC-SVM optimization problem:

$$\begin{aligned} \min_{\boldsymbol{\alpha}} \quad & \frac{1}{2} \sum_{i=0}^n \sum_{j=0}^n \alpha_i \alpha_j K(\mathbf{x}_i, \mathbf{x}_j) \\ \text{subject to} \quad & 0 \leq \alpha_i \leq \frac{1}{\nu n}, \quad i = 1, \dots, n \\ & \sum_{i=0}^n \alpha_i = 1 \end{aligned} \tag{3.1.3}$$

Note that compared to the original Wolfe Dual form, (2.1.10), our new dual form no longer contains the term $\sum_{i=0}^n \alpha_i$ in part because our one-class assumption has altered the constraints to include $\sum_{i=0}^n \alpha_i = 1$. The problem is also now minimization which removes the negative sign we had also seen in the standard SVM case (2.1.10).

Due to the similar formulation of the OC-SVM to the standard SVM we can use a slightly modified version of the SMO algorithm to train OC-SVMs. One example of the SMO algorithm modified for OC-SVMs can be found in [14].

With this dual formulation, the decision function for the OC-SVM for $\boldsymbol{\alpha}$ and ρ who solve (3.1.3) is:

$$f(x) = \sum_{i=1}^n \alpha_i K(\mathbf{x}_i, \mathbf{x}) - \rho$$

3.2 ELLIPSOIDAL SUPPORT VECTOR MACHINES

Our next component to consider when working towards the OC-CESVM is the ellipsoidal aspect. The Ellipsoidal Support Vector Machine (e-SVM) was first introduced in [10] as a modification of the traditional SVM.

To understand the e-SVM it helps to describe the **version space**. The version space is the set of all possible hypothesis functions (classification rules) that are consistent with the training data. More formally,

Definition (Version Space): Given a set of training data, $\mathcal{D}$, and the *Hypothesis Space*, $\mathcal{H}$, which is the space of all hypotheses (or decision rules) over the dataset, we define the version space $V(\mathcal{D})$ as follows:

$$V(\mathcal{D}) = \{h \in H | \forall i = \{1 \dots n\} : h(\mathbf{x}_i) = y_i\}$$

The **Bayes Point** is the center of mass of the version space – that is, the Bayes Point is the average behavior of all hypotheses in the version space. In most cases, the Bayes Point cannot be directly computed so methods of approximating the Bayes Point include algorithms like the Bayes Point Machine introduced in [3].

Each hypothesis in the version space is defined by its weight vector $\mathbf{w}$. As the SVM for the dataset looks to maximize the distance between the margin and the nearest data points, the weight vector for the SVM corresponds to the center of the largest sphere inscribed in the version space. The weight vector of our SVM, thus, is an approximation of the Bayes Point itself.

The e-SVM scheme modifies this so that we are looking at the largest inscribed ellipsoid in the version space instead of sphere. While one could define an e-SVM (or a SVM for that matter) similarly to a Bayes Point Machine but it makes much more practical sense to define it using the SVM framework discussed earlier. Bayes Point Machine formulation involves estimating posterior probabilities of each hypothesis which is quite costly whereas we can reap the benefits of the SMO algorithm and other optimizations found for standard SVMs if we formulate the e-SVM similarly.

Here I will detail a few of the steps of the formulation of the e-SVM – for more specifics, the derivation can be found in [10]. For reference, the following is the optimization problem for finding the maximum soft-margin hypersphere in the version space:

$$\begin{aligned} \max_{\rho, \mathbf{w}, b, \zeta} \quad & -n\rho + \frac{1}{\nu} \sum_{i=1}^n \zeta_i \\ \text{subject to} \quad & y_i(\mathbf{x}_i^\top \mathbf{w} + b) + \|\mathbf{x}_i\|_2^2 \zeta_i \geq \|\mathbf{x}_i\|_2^2 \rho, \\ & \|\mathbf{w}\|_2 \leq 1, \quad i = 1, \dots, n \end{aligned} \tag{3.2.1}$$

Note that $\nu > 0$ is a given constant. We can formulate an ellipsoid centered at $\mathbf{w}$ with $\mathcal{E} = \{\mathbf{E}\mathbf{u} + \mathbf{w} \mid \|\mathbf{u}\|_2 \leq 1, \mathbf{E} \succeq 0\}$. Thus, to transition from this formulation with a hypersphere to that of a hyperellipsoid we alter the constraints of (3.2.1) as follows:

$$y_i(\mathbf{x}_i^\top (\mathbf{E}\mathbf{u} + \mathbf{w}) + b) + \|\mathbf{x}_i\|_2^2 \zeta_i \geq \|\mathbf{x}_i\|_2^2 \rho, \quad \forall \mathbf{u}, \|\mathbf{u}\|_2 \leq 1$$

From here the problem can be optimized for lower compute complexity to land on the final formulation of the primal soft-margin hyperellipsoidal SVM:

$$\begin{aligned} \min_{\mathbf{B}, \boldsymbol{\xi}, \mathbf{w}, b} \quad & -r \log |\mathbf{B}| + (1-r) \text{tr}(\mathbf{B}) + \frac{1}{\nu} \sum_{i=1}^n \frac{1}{\|\mathbf{x}_i\|_2^2} \xi_i \\ \text{subject to} \quad & y_i(\mathbf{x}_i^\top \mathbf{w} + b) + \xi_i \geq \mathbf{x}_i^\top \mathbf{B} \mathbf{x}_i, \\ & \|\mathbf{w}\|_2 \leq 1, \quad \boldsymbol{\xi} \geq 0, \quad \mathbf{B} \succeq 0, \quad i = 1, \dots, n \end{aligned} \quad (3.2.2)$$

In this equation r and ν are given constants. We assume the matrix $\mathbf{E}$ can be written as $\mathbf{E} = \mathbf{E}_0 + \mathbf{B}$ where $\mathbf{E}_0$ is the current solution and $\mathbf{B}$ is a deviation from it, then $\xi_i = \|\mathbf{E}_0 \mathbf{x}_i\|_2^2 \zeta_i$. We minimize the determinant of $\mathbf{B}$ because this is analogous to maximizing the volume of the hyperellipsoid.

The non-kernelized dual form of this problem is as follows:

$$\begin{aligned} \max_{\boldsymbol{\alpha}, \gamma} \quad & r \log |\mathbf{B}^{-1}| - \frac{1}{4\gamma} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j - \gamma \\ \text{subject to} \quad & \mathbf{B}^{-1} = \frac{1}{r} \left((1-r) \mathbf{I} + \sum_{i=1}^n \alpha_i \mathbf{x}_i \mathbf{x}_i^\top \right) \\ & \sum_{i=1}^n \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq \frac{1}{\|\mathbf{x}_i\|_2^2 \nu}, \quad \gamma > 0 \end{aligned} \quad (3.2.3)$$

From the presence of data points $\mathbf{x}_i$ only when multiplied together, we can see how one would kernelize this problem. Let $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ and $\mathbf{k}_i$ be the i th column of the kernel matrix, K with entries K_{ij} . The kernelized dual form is:

$$\begin{aligned} \max_{\boldsymbol{\alpha}, \gamma} \quad & r \log \left| \mathbf{I} + \frac{1}{1-r} \sum_{i=1}^n \alpha_i \mathbf{k}_i \mathbf{e}_i^\top \right| - \frac{1}{4\gamma} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K_{ij} - \gamma \\ \text{subject to} \quad & \sum_{i=1}^n \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq \frac{1}{\|\mathbf{x}_i\|_2^2 \nu}, \quad \gamma \geq 0 \end{aligned} \quad (3.2.4)$$

3.3 ONE-CLASS CENTERED ELLIPSOIDAL SUPPORT VECTOR MACHINES

Now that we have looked at the one-class SVMs and the ellipsoidal SVMs individually, we turn to the combined one-class centered ellipsoidal SVM. We will be looking at the formulation of

the OC-CESVM presented in [12].

FINDING THE PRIMAL PROBLEM

As a reminder our data is of the form $\mathcal{D} = \{\mathbf{x}_i | \mathbf{x}_i \in \mathbb{R}^p\}_{i=1}^n$ in the one-class case. We start by labeling our feature space, $\mathbb{R}^f$, as well as the function that maps our data into the feature space, $\phi : \mathbb{R}^p \rightarrow \mathbb{R}^f$. Our goal is to find an ellipsoid in the feature space with the minimum effective radius R that encloses the majority of the data. We define:

$$\Sigma = \frac{1}{n} \sum_{i=1}^n (\phi(\mathbf{x}_i) - \mu)^\top (\phi(\mathbf{x}_i) - \mu), \quad \mu = \frac{1}{n} \sum_{i=1}^n \phi(\mathbf{x}_i)$$

The use of μ in the definition of Σ functions to center our ellipsoid around the mean of the data. The primal form of the OC-CESVM optimization problem is as follows:

$$\begin{aligned} \min_{R, \xi} \quad & R^2 + \frac{1}{\nu n} \sum_{i=0}^n \xi_i \\ \text{subject to} \quad & \phi(\mathbf{x}_i) \Sigma^{-1} \phi(\mathbf{x}_i)^\top \leq R^2 + \xi_i, \quad i = 1, \dots, n \\ & \xi_i \geq 0, \quad i = 1, \dots, n \end{aligned} \tag{3.3.1}$$

We can see that this formulation is really quite similar to the standard one-class SVM formulation (3.1.1) but we are minimizing $R^2 \in \mathbb{R}^+$ rather than $\|\mathbf{w}\|^2$. We crucially change our constraints to represent a hyperellipsoid rather than a hyperplane by using $\phi(\mathbf{x}_i) \Sigma^{-1} \phi(\mathbf{x}_i)^\top \leq R^2 + \xi_i$ instead of the more familiar hyperplane equation $\langle \mathbf{w}, \phi(\mathbf{x}_i) \rangle \geq \rho - \xi_i$.

Now, for the same reason as with the standard SVM we aim to formulate this optimization problem without the need to explicitly define ϕ . We look to employ a kernel, $k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x}) \phi(\mathbf{x}')^\top$, and we will define our kernel matrix, $K \in \mathbb{R}^{n \times n}$ where $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$. Because we are centering our data in the feature space by subtracting the means, μ , we will also have a centered kernel matrix $\tilde{K}$. We can express this centered kernel matrix in terms of the original:

$$\tilde{K} = K - \mathbf{1}_n K - K \mathbf{1}_n + \mathbf{1}_n K \mathbf{1}_n$$

Where $\mathbf{1}_n$ is the $n \times n$ matrix with each entry equal to $\frac{1}{n}$. This process is referred to as **double centering** the matrix K . We see in [12] that $\tilde{K}$ is positive semi-definite and we can write $\tilde{K} = P \Lambda P^\top$, where Λ is a diagonal matrix with the positive eigenvalues along the diagonal and P is the eigenvector matrix corresponding to the positive eigenvalues. Then it can be shown that we can write:

$$\phi(\mathbf{x}_i) \Sigma^{-1} \phi(\mathbf{x}_i)^\top = \|\sqrt{n} \Lambda^{-1} P^\top \tilde{K}^i\|^2$$

This means that we can now rewrite the constraints of our optimization problem (3.3.1) as follows:

$$\begin{aligned} \min_{R, \xi} \quad & R^2 + \frac{1}{\nu n} \sum_{i=0}^n \xi_i \\ \text{subject to} \quad & \|\sqrt{n} \Lambda^{-1} P^\top \tilde{K}^i\|^2 \leq R^2 + \xi_i, \quad i = 1, \dots, n \\ & \xi_i \geq 0, \quad i = 1, \dots, n \end{aligned} \tag{3.3.2}$$

Now when we define our kernel function, we no longer need to explicitly define the transformation ϕ into the feature space as our constraints are all in terms of the kernel matrix.

FINDING THE DUAL PROBLEM

We now look to find the dual representation of (3.3.2). First, we formulate the Lagrangian for this problem:

$$\mathcal{L}(R^2, \xi, \alpha, \gamma) = R^2 + \frac{1}{\nu n} \sum_{i=0}^n \xi_i - \sum_{i=0}^n \gamma_i \xi_i - \sum_{i=0}^n \alpha_i [R^2 - \|\sqrt{n}\Lambda^{-1}P^\top \tilde{K}^i\|^2 + \xi_i]$$

First we will differentiate with respect to R^2 and impose stationarity:

$$\frac{\partial \mathcal{L}(R^2, \xi, \alpha, \gamma)}{\partial R^2} = 1 - \sum_{i=1}^n \alpha_i = 0 \implies \sum_{i=1}^n \alpha_i = 1$$

Now with respect to ξ_i :

$$\frac{\partial \mathcal{L}(R^2, \xi, \alpha, \gamma)}{\partial \xi_i} = \frac{1}{\nu n} - \sum_{i=1}^n \gamma_i - \sum_{i=1}^n \alpha_i = 0 \implies \gamma_i = \frac{1}{\nu n} - \alpha_i \quad \forall i = 1, \dots, n$$

We require our Lagrange multipliers to be non-negative to satisfy the KKT Conditions so $\alpha_i \geq 0$ and $\gamma_i \geq 0$. Combining this with $\gamma_i = \frac{1}{\nu n} - \alpha_i$ gives us:

$$0 \leq \alpha_i \leq \frac{1}{\nu n}$$

Now we rearrange $\mathcal{L}(R^2, \xi, \alpha, \gamma)$:

$$\begin{aligned} \mathcal{L}(R^2, \xi, \alpha, \gamma) &= R^2 + \frac{1}{\nu n} \sum_{i=0}^n \xi_i - \sum_{i=0}^n \gamma_i \xi_i - \sum_{i=0}^n \alpha_i R^2 + \sum_{i=0}^n \alpha_i \|\sqrt{n}\Lambda^{-1}P^\top \tilde{K}^i\|^2 - \sum_{i=0}^n \alpha_i \xi_i \\ &= R^2 + \left[\frac{1}{\nu n} - \sum_{i=1}^n \gamma_i - \sum_{i=1}^n \alpha_i \right] \sum_{i=0}^n \xi_i - R^2 \sum_{i=0}^n \alpha_i + \sum_{i=0}^n \alpha_i \|\sqrt{n}\Lambda^{-1}P^\top \tilde{K}^i\|^2 \end{aligned}$$

Substitute in $\sum_{i=1}^n \alpha_i = 1$ and $\frac{1}{\nu n} - \sum_{i=1}^n \gamma_i - \sum_{i=1}^n \alpha_i = 0$:

$$R^2 - R^2 + \sum_{i=0}^n \alpha_i \|\sqrt{n}\Lambda^{-1}P^\top \tilde{K}^i\|^2$$

Thus,

$$\mathcal{L}(\alpha) = \sum_{i=0}^n \alpha_i \|\sqrt{n}\Lambda^{-1}P^\top \tilde{K}^i\|^2$$

We can now formulate the dual form of this problem:

$$\begin{aligned}
& \max_{\boldsymbol{\alpha}} \quad - \sum_{i=1}^n \alpha_i \|\sqrt{n} \Lambda^{-1} P^\top \tilde{K}^i\|^2 \\
& \text{subject to} \quad 0 \leq \alpha_i \leq \frac{1}{\nu n}, \quad i = 1, \dots, n \\
& \quad \sum_{i=1}^n \alpha_i = 1
\end{aligned} \tag{3.3.3}$$

This dual problem is a linear optimization problem. In lieu of the SMO algorithm, existing linear optimization techniques are typically used to solve this problem.

As we have seen before, any given data point $\mathbf{x}_i$ is a support vector if $\alpha_i > 0$. Moreover, it is an anomaly if $\alpha_i = \frac{1}{\nu n}$ as these points lie outside the hyperellipsoid. Finally, if $0 < \alpha_i < \frac{1}{\nu n}$, the point is called a boarder support vector and lies on the hyperellipsoid.

3.4 VARIATIONS ON THE SMO ALGORITHM

Before discussing alterations commonly made to the SMO algorithm to accommodate for either one-class SVMs or ellipsoidal SVMs we note an optimization for the algorithm itself proposed by [5].

In [5], Keerthi et al identify inefficiency in how the bias, b , is calculated with respect to the current $\boldsymbol{\alpha}$ values. They find that while optimal $\boldsymbol{\alpha}$ may be reached, the algorithm can overlook this solution due to an incorrect b value. To rectify this inefficiency they remove the computation of b entirely.

They note that the duality gap of our optimization problem is only zero at the optimum and otherwise the objective function value of the primal solution is greater than or equal to the objective function value of the dual solution. For a given $\boldsymbol{\alpha}$, $\boldsymbol{\xi}$ can be chosen optimally, as a function of b , to minimize the duality gap. Thus, they find and use the value of b that minimizes the duality gap.

A second optimization proposed by Keerthi et al impacts the heuristics for selecting α_a and α_b . Instead of selecting α_b based solely on if it violates the KKT conditions, they look for the point that violates the KKT conditions the most.

Both modified versions of the SMO algorithm, one for the one-class SVM and one for the ellipsoidal SVM, adopt the optimizations proposed in [5] before making their modifications.

Proposed in [14], there is an existing version of the SMO algorithm formulated to solve the one-class SVM problem. To modify the SMO algorithm accordingly, they go through the same steps of deriving the closed form solution directly for optimizing two Lagrange multipliers but

in the one-class case. Where we previously had (2.3.3):

$$\alpha_b^* = \alpha_b + \frac{y_b[(f(\mathbf{x}_a) - y_a) - (f(\mathbf{x}_b) - y_b)]}{K_{aa} + K_{bb} - 2K_{ab}}$$

Remember that $f(\mathbf{x}_i) = \sum_{j=1}^n \alpha_j y_j K_{ij} + b$.

The new step for α_b^* becomes:

$$\alpha_b^* = \alpha_b + \frac{(K_{aa}\alpha_a + K_{ba}\alpha_b + \sum_{j=1, j \neq a, b}^n \alpha_j K_{aj}) - (K_{ab}\alpha_a + K_{bb}\alpha_b + \sum_{j=1, j \neq a, b}^n \alpha_j K_{bj})}{K_{aa} + K_{bb} - 2K_{ab}}$$

From here they use very similar heuristics to the original SMO algorithm and have the same overall structure.

The modification of the SMO algorithm for the ellipsoidal SVM is proposed in [10]. At a high level, there are three main changes to the SMO algorithm in the e-SVM case: normalize $\|\mathbf{w}\|_2 = 1$, optimize the step size, and alter the KKT conditions. Another complication of the ellipsoidal case is that the matrix $\mathbf{B}$ must be updated to perform the check on KKT violation of the data points. More specifics of the alterations to this algorithm can be found in [10].

4 DATA PREPARATION

When working with datasets like LUCE, there are a few steps one must take in order to prepare the data for a machine learning model like a SVM. For this dataset, and many WSN datasets, dimensionality reduction can greatly improve the speed of anomaly detection to allow for more efficient online algorithms. In this section we will talk about data preparation and dimensionality reduction techniques commonly used.

4.1 NORMALIZATION AND DATA CLEANING

When working with data like the LUCE dataset, we first divide the dataset into three distinct sets: the training set, the validation set, and the test set. This allows us to initially train our machine learning model – in this case our SVM – on the training set and then use the validation set to tune hyperparameters like the ν value for one-class SVMs or the C value for standard SVMs.

Dividing the dataset is also an important step before normalization because we will be using the mean and standard deviation of the training dataset to normalize all three subsets. That is, we will first subtract the training mean from each data point and then divide by the training standard deviation.

This recreates the real-world preprocessing of live WSN data in which we would store the training mean and standard deviation along with the Lagrange multipliers of our SVM and the

kernel function in the sensor nodes of the WSN. These values (and function) are all that we actually require to run anomaly detection on the incoming WSN data.

Another important aspect of data preprocessing is deciding how you will handle sensor dropout, or NaN values, in the dataset. In the context of the LUCE dataset where we are looking at frequent WSN readings, when even one sensor drops out the rest of the data from that time cannot be used either – the $\mathbf{x}_i$ vector would no longer have the correct dimension. Luckily, the LUCE dataset does not have extremely frequent instances of sensor dropout with only about 1.017 percent of the dataset being NaN readings. This allowed us to make the choice to drop all readings from times that included at least one NaN sensor reading. When sensor dropout is much more frequent, techniques that involve estimating the missing values can be used instead.

The code that I wrote to load the LUCE dataset and do this preprocessing can be seen in cf. [Appendix D](#).

4.2 PRINCIPAL COMPONENT ANALYSIS AND VARIATIONS

When the goal is to create a lightweight and fast anomaly detection system for a WSN, dimensionality reduction can be one of the best tools to use. Reducing the dimension of the data before attempting to do training or inference with the SVM allows for lower computational complexity and better memory utilization [16].

Datasets are composed of vectors, $\mathbf{x}_i \in \mathbb{R}^p$, where each component $\mathbf{x}_{ij}$, $\forall j$ st. $1 \leq j \leq p$ is called a feature or a variable. Principal Component Analysis (PCA) is a common dimensionality reduction technique that transforms uncorrelated features into a condensed (smaller) set of correlated features. To do this one computes the covariance matrix of the dataset and finds eigenvectors as well as eigenvalues to determine *principal components*.

Incremental PCA is a variant of PCA that conducts PCA on sequential subsets of the data instead of on the entire dataset at one time. Doing PCA incrementally is more space efficient and allows for the easier handling of high-dimensional or very large datasets. Candid Covariance-Free Incremental PCA (CCIPCA), introduced in [15], is a version of incremental PCA that avoids computing the entire covariance matrix, instead relying on statistical efficiency.

5 FUTURE WORK

The CESVM-DR anomaly detection scheme was designed for lightweight online anomaly detection in WSNs but could theoretically be applied to any hardware and memory constrained environment. As a continuation of this report and inspired by my role as the payload lead on Columbia University’s rocketry team, I intend to deploy a OC-CESVM that takes in only three sensor readings (thus avoiding the need for PCA or CCIPCA) to do anomaly detection on

the typical rocket flight path. The environment will be particularly noisy and prone to sensor dropout which creates an interesting complication.

In this context, I will be looking at time-series data for acceleration, velocity, and altitude, hoping to identify anomalous flight patterns for the rocket like the engine sputtering or a parachute failure. Another interesting aspect of this situation is that the SVM will be trained on simulated test data as we do not have access to real flight data for the engine we will be using. This opens up an interesting avenue of study into the validity of simulated data for training SVMs and “how close” we can get the simulations to replicating the real thing.

Our rocket will be launching in late May and we look forward to seeing how our specialized OC-CESVM will generalize to applications in aerospace rather than just WSNs.

Another avenue for potential continuation is in the development of a variant of the SMO algorithm for the OC-CESVM. There are current formulations for the SMO algorithm to solve both the one-class SVM and the ellipsoidal SVM but it has yet to be defined for the specific case of the OC-CESVM. The SMO algorithm is more efficient in the case of each SVM variant so it would make sense for it to also be an improvement on existing optimization techniques.

APPENDIX A: MERCER'S THEOREM

Let X be a compact subset of $\mathbb{R}^n$. Suppose K is a continuous symmetric function such that the integral operator $T_K : L_2(X) \rightarrow L_2(X)$,

$$(T_K f)(\cdot) = \int_X K(\cdot, \mathbf{x}) f(\mathbf{x}) d\mathbf{x}$$

is positive, that is

$$\int_{X \times X} K(\mathbf{x}, \mathbf{z}) f(\mathbf{x}) f(\mathbf{z}) d\mathbf{x} d\mathbf{z} \geq 0$$

for all $f \in L_2(X)$. Then we can expand $K(\mathbf{x}, \mathbf{z})$ in a uniformly convergent series (on $X \times X$) in terms of the eigenvectors of T_K , $\phi_j \in L_2(X)$, normalized in such a way that $\|\phi_j\|_{L_2} = 1$, and positive associated eigenvalues $\lambda_j \geq 0$,

$$K(\mathbf{x}, \mathbf{z}) = \sum_{j=1}^{\infty} \lambda_j \phi_j(\mathbf{x}) \phi_j(\mathbf{z})$$

This formulation of Mercer's Theorem was presented in [2]. The theorem was originally introduced in [9].

APPENDIX B: THE SEQUENTIAL MINIMAL OPTIMIZATION (SMO) ALGORITHM

This implementation of the SMO algorithm is based on the pseudocode given by John Platt when he initially introduced the algorithm in [11].

```
1 #Throughout I will use 1 and 2 instead of a and b
2 #That is: alpha_a := alpha1, alpha_b := alpha2, etc.
3 import numpy as np
4 import random
5
6 class SMO:
7     def __init__(self, kernel, X, y, C):
8         self.kernel = kernel
9         self.X = X
10        self.y = y
11        self.C = C
12        #suggested tolerance by Cristianini and Shawe-Taylor (2000)
13        #suggested epsilon from original SMO paper
14        self.tol = 1e-2
15        self.eps = 1e-3
16
17    def update_b(self, a1, alpha1, a2, alpha2, y1, y2, E1, E2, k11, k12, k22,
18    oldb):
19        b1 = E1 + y1*(a1 - alpha1)*k11 + y2*(a2 - alpha2)*k12 + oldb
20        b2 = E2 + y1*(a1 - alpha1)*k12 + y2*(a2 - alpha2)*k22 + oldb
21
22        if 0 < a1 and a1 < self.C:
23            return b1
24        elif 0 < a2 and a2 < self.C:
25            return b2
26        else:
27            return (b1 + b2)/2
28
29    def secondChoiceHeuristic(self, E2, index2, E, alpha):
30        non_bound = [i for i in range(len(alpha)) if 0 < alpha[i] < self.C and
31        i != index2]
32        if not non_bound:
33            return None
34        if E2 < 0:
35            return max(non_bound, key=lambda i: E[i])
36        else:
37            return min(non_bound, key=lambda i: E[i])
38
39    def stepIndex(self, index, length):
40        if index == length -1:
41            index = 0
42        else:
43            index += 1
44        return index
45
46    def objectiveFunction(self, alpha2, index2, alpha):
47        oldVal = alpha[index2] #temporarily alter alpha
48        alpha[index2] = alpha2
49        otherSum = 0
```

```

48     for i in range(self.y.size):
49         for j in range(self.y.size):
50             otherSum += alpha[i]*alpha[j]*self.y[i]*self.y[j]*self.kernel(
self.X[i], self.X[j])
51
52     obj = sum(alpha) - (1/2)*otherSum
53
54     alpha[index2] = oldVal #revert alpha
55     return obj
56
57 def takeStep(self, index1, index2, alpha, b, E):
58     if index1 == index2:
59         return 0, alpha, b, E
60     alpha1 = alpha[index1]
61     alpha2 = alpha[index2]
62     y1 = self.y[index1]
63     y2 = self.y[index2]
64     E1 = E[index1]
65     E2 = E[index2]
66     s = y1 * y2
67     #From the report notation: L = U and H = V
68     if y1 == y2:
69         U = max(0, alpha1 + alpha2 - self.C)
70         V = min(self.C, alpha1 + alpha2)
71     else:
72         U = max(0, alpha1 - alpha2)
73         V = min(self.C, self.C - alpha1 + alpha2)
74
75     if U == V:
76         return 0, alpha, b, E
77
78     k11 = self.kernel(self.X[index1], self.X[index1])
79     k12 = self.kernel(self.X[index1], self.X[index2])
80     k22 = self.kernel(self.X[index2], self.X[index2])
81
82     eta = 2*k12 - k11 - k22
83     #clip the value
84     if eta < 0:
85         a2 = alpha2 - (y2 * (E1-E2))/eta
86         if a2 < U:
87             a2 = U
88         elif a2 > V:
89             a2 = V
90     else:
91         #objective function at a2 = U
92         Uobj = self.objectiveFunction(U, index2, alpha)
93         #at a2 = V
94         Vobj = self.objectiveFunction(V, index2, alpha)
95
96         if Uobj > Vobj+self.eps:
97             a2 = U
98         elif Uobj < Vobj-self.eps:
99             a2 = V
100        else:
101            a2 = alpha2
102
103     if abs(a2 - alpha2) < self.eps*(a2 + alpha2 + self.eps):

```

```

104         return 0, alpha, b, E
105
106     a1 = alpha1 + s*(alpha2 - a2)
107
108     #same condition down here for a1
109     if abs(a1 - alpha1) < self.eps*(a1 + alpha1 + self.eps):
110         return 0, alpha, b, E
111
112     #we may need to also clip a1
113     if a1 < 0:
114         a1 = 0
115     elif a1 > self.C:
116         a1 = self.C
117
118     #update alpha vector
119     alpha[index1] = a1
120     alpha[index2] = a2
121     b_old = b
122     b = self.update_b(a1, alpha1, a2, alpha2, y1, y2, E1, E2, k11, k12, k22
, b)
123
124     #Update error cache using new Lagrange multipliers and b value
125     for i in range(len(E)):
126         E[i] += (
127             y1 * (a1 - alpha1) * self.kernel(self.X[index1], self.X[i])
128             + y2 * (a2 - alpha2) * self.kernel(self.X[index2], self.X[i])
129             + (b - b_old)
130         )
131     #Force exact zeros for updated points
132     E[index1] = 0.0
133     E[index2] = 0.0
134     return 1, alpha, b, E
135
136 def examineExample(self, index2, alpha, b, E):
137     y2 = self.y[index2]
138     alpha2 = alpha[index2]
139     E2 = E[index2]
140     r2 = E2 * y2
141
142     if (r2 < -self.tol and alpha2 < self.C) or (r2 > self.tol and alpha2 >
0):
143         if len([a for a in alpha if 0 < a < self.C]) > 1:
144             index1 = self.secondChoiceHeuristic(E2, index2, E, alpha)
145             if index1 is not None:
146                 val, alpha, b, E = self.takeStep(index1, index2, alpha, b,
E)
147                 if val:
148                     return 1, alpha, b, E
149
150     #loop through alpha values on boundary starting at random index
151     current = random.randint(0, self.y.size-1) #inclusive
152     start = current
153     #check the first one before the loop
154     if alpha[current] != 0 and alpha[current] != self.C:
155         val, alpha, b, E = self.takeStep(current, index2, alpha, b, E)
156         if val:
157             return 1, alpha, b, E
158     else:

```

```

158         current = self.stepIndex(current, self.y.size)
159
160     while True:
161         if current == start:
162             break
163
164         if alpha[current] != 0 and alpha[current] != self.C:
165             val, alpha, b, E = self.takeStep(current, index2, alpha, b,
E)
166
167             if val:
168                 return 1, alpha, b, E
169             else:
170                 current = self.stepIndex(current, self.y.size)
171
172     #otherwise, loop over all values starting at a random index
173     current = random.randint(0, self.y.size-1) #inclusive
174     start = current
175
176     val, alpha, b, E = self.takeStep(current, index2, alpha, b, E)
177     while not val:
178         current = self.stepIndex(current, self.y.size)
179         if current == start:
180             break
181         val, alpha, b, E = self.takeStep(current, index2, alpha, b, E)
182
183     if val:
184         return 1, alpha, b, E
185
186     return 0, alpha, b, E
187
188 def run_smo(self):
189     pointCount = self.y.size
190     alpha = np.zeros(pointCount)
191     numChanged = 0
192     examineAll = 1
193     b = 0
194     #error cache
195     E = -self.y.astype(float) #because alpha and b are all zeroes
196
197     while numChanged > 0 or examineAll:
198         numChanged = 0
199         if examineAll:
200             for i in range(pointCount):
201                 num, alpha, b, E = self.examineExample(i, alpha, b, E)
202                 numChanged += num
203         else:
204             for i in range(pointCount):
205                 if alpha[i] != 0 and alpha[i] != self.C:
206                     num, alpha, b, E = self.examineExample(i, alpha, b, E)
207                     numChanged += num
208         if examineAll == 1:
209             examineAll = 0
210         elif numChanged == 0:
211             examineAll = 1
212     return alpha, b

```

APPENDIX C: SVM CLASS USING SMO ALGORITHM

Here is code that defines a SVM class using the SMO algorithm implemented in [Appendix B](#).

```
1 from src.models.standard_smo import SMO
2
3 class SVM:
4     """
5     SVM implementation
6     """
7     def __init__(self, kernel, gamma, C):
8         """
9         Args:
10             kernel (function)
11             nu (float): An upper bound on the fraction of anomalies.
12             gamma (str or float): Kernel coefficient.
13         """
14         self.kernel = kernel
15         self.gamma = gamma
16         self.C = C
17         #below not yet used - need to train on data first
18         self.alpha, self.beta = 0, 0
19         self.X_train, self.y_train = 0, 0
20         self.fitted = False
21
22     def fit(self, X_train, y_train):
23         self.X_train = X_train
24         self.y_train = y_train
25         smo = SMO(self.kernel, X_train, y_train, self.C)
26         self.alpha, self.beta = smo.run_smo()
27
28         self.fitted = True
29
30     def pred_one(self, x):
31         result = 0
32         #  $f(x) = \sum_{i=1}^n [\alpha_i * y_i * K(x_i, x)]$ 
33         for i in range(len(self.alpha)):
34             result += self.alpha[i] * self.y_train[i] * self.kernel(self.
X_train[i], x)
35         result += self.beta
36         return 1 if result >= 0 else -1
37
38     def predict(self, X_test):
39         """Predict normal (+1) or anomaly (-1) for test data. (Run inference on
the test data)"""
40         if not self.fitted:
41             print("You must train your SVM before predicting with it!")
42             return
43
44         n, _ = X_test.shape
45         preds = [0]*n
46         for i in range(n):
47             preds[i] = self.pred_one(X_test[i])
48         return preds
```

APPENDIX D: LUCE PREPROCESSING

This is the code I used to process the LUCE dataset and prepare it for ingestion into a one-class SVM variant.

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import sys, os
5 sys.path.append(os.path.abspath("..")) #need to do this to find src when
   launching from notebook
6 from src.data.pca import library_pca
7
8 def normalize(data, mean, std_dev):
9     data = data - mean
10    data = data / std_dev
11    return data
12
13
14 def preprocess(train, val, eval):
15     ##### Standardization
16     train_mean = np.nanmean(train, axis=0) #need to exclude NaN values
17     train_std_dev = np.nanstd(train, axis=0) #same here
18     train = normalize(train, train_mean, train_std_dev)
19
20     #now I will normalize the validation and evaluation sets with the mean and
   stdd from the test set
21     val = normalize(val, train_mean, train_std_dev)
22     eval = normalize(eval, train_mean, train_std_dev)
23
24     #lets count how many NaN values we have before we get rid of them
25     nan_train = np.sum(np.isnan(train))
26     nan_val = np.sum(np.isnan(val))
27     nan_eval = np.sum(np.isnan(eval))
28     print("Number of NaN values:")
29     print(f"Training:    {nan_train} out of {train.size}  -- that is {round((
   nan_train/train.size) * 100, 4)} percent.")
30     print(f"Validation:  {nan_val} out of {val.size}    -- that is {round((
   nan_val/val.size) * 100, 4)} percent.")
31     print(f"Evaluation:  {nan_eval} out of {eval.size}  -- that is {round((
   nan_eval/eval.size) * 100, 4)} percent.")
32     print(f"Overall:    {nan_eval + nan_train + nan_val} out of {val.size +
   train.size + eval.size}  -- that is {round(((nan_eval + nan_train + nan_val
   )/(val.size + train.size + eval.size)) * 100, 4)} percent.")
33
34     #before I can do PCA I need to get rid of NaN values
35     train = train[~np.isnan(train).any(axis=1)]
36     val = val[~np.isnan(val).any(axis=1)]
37     eval = eval[~np.isnan(eval).any(axis=1)]
38
39     ##### PCA
40     pca_er = library_pca(train)
41     train = pca_er.transform(train)
42     val = pca_er.transform(val)
43     eval = pca_er.transform(eval)
```

```

44
45     return train, val, eval
46
47
48 def ibrl():
49     file_path = '../data/IBRLdata.txt' #NOTE: This may need to back out of the
50     directory with ../ depending on where I run this python file from
51     #there are 2313682 lines in this file, each representing a set of readings
52     #BUT we only have full readings until line 2313153
53
54     #I want to make a training dataset, a validation dataset, and an evaluation
55     dataset
56     #they will be 70% training, 15% validation, and 15% evaluation
57     train_size = int(0.7 * 2313153)
58     val_size = int(0.15 * 2313153)
59     eval_size = int(0.15 * 2313153) #allows us to not include the end values
60     that don't have readings
61
62     #format:
63     #date:yyyy-mm-dd time:hh:mm:ss.xxx epoch:int moteid:int temperature:real
64     humidity:real light:real voltage:real
65     df = pd.read_csv(file_path, delimiter=" ", header=None)
66     #only use sensor columns for PCA + SVM (real numbers)
67     X = df.iloc[:, -4:].to_numpy()
68
69     #NOTE: I do not need the time data... because I am classifying outliers in
70     the usual values
71
72     X_train = X[:train_size]
73     X_val = X[train_size:train_size+val_size]
74     X_eval = X[train_size+val_size:train_size+val_size+eval_size+1]
75
76     return X_train, X_val, X_eval

```

BIBLIOGRAPHY

- [1] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Mach. Learn.*, 20(3):273–297, September 1995. doi:[10.1023/A:1022627411411](https://doi.org/10.1023/A:1022627411411).
- [2] Nello Cristianini and John Shawe-Taylor. *An Introduction to Support Vector Machines and Other Kernel-based Learning Methods*. Cambridge University Press, 2000.
- [3] Ralf Herbrich, Thore Graepel, and Colin Campbell. Bayes point machines. *J. Mach. Learn. Res.*, 1:245–279, September 2001. doi:[10.1162/153244301753683717](https://doi.org/10.1162/153244301753683717).
- [4] S. R. Jino Ramson and D. Jackuline Moni. Applications of wireless sensor networks — a survey. In *2017 International Conference on Innovations in Electrical, Electronics, Instrumentation and Media Technology (ICEEIMT)*, pages 325–329, 2017. doi:[10.1109/ICIEEIMT.2017.8116858](https://doi.org/10.1109/ICIEEIMT.2017.8116858).
- [5] S. S. Keerthi, S. K. Shevade, C. Bhattacharyya, and K. R. K. Murthy. Improvements to platt’s smo algorithm for svm classifier design. *Neural Computation*, 13(3):637–649, 2001. doi:[10.1162/089976601300014493](https://doi.org/10.1162/089976601300014493).
- [6] Alexandre Kowalczyk. *Support Vector Machines Succinctly*. Syncfusion, 2017. URL: <https://www.syncfusion.com/succinctly-free-ebooks/support-vector-machines-succinctly>.
- [7] H. W. Kuhn and A. W. Tucker. Nonlinear programming. In *Proceedings of the Second Berkeley Symposium on Mathematical Statistics and Probability, 1950*, pages 481–492, Berkeley and Los Angeles, 1951. University of California Press.
- [8] Lausanne urban canopy experiment. <http://lcav.epfl.ch/cms/lang/en/pid/86035>. Accessed: 20 November 2025.
- [9] James Mercer. Xvi. functions of positive and negative type, and their connection the theory of integral equations. *Philosophical Transactions of the Royal Society of London, Series A: Containing Papers of a Mathematical or Physical Character*, 209(441-458):415–446, 01 1909. arXiv:<https://royalsocietypublishing.org/rsta/article-pdf/209/441-458/415/239856/rsta.1909.0016.pdf>, doi:[10.1098/rsta.1909.0016](https://doi.org/10.1098/rsta.1909.0016).
- [10] Michinari Momma, Kohei Hatano, and Hiroki Nakayama. Ellipsoidal support vector machines. In Masashi Sugiyama and Qiang Yang, editors, *Proceedings of 2nd Asian Conference on Machine Learning*, volume 13 of *Proceedings of Machine Learning Research*, pages

31–46, Tokyo, Japan, 08–10 Nov 2010. PMLR. URL: <https://proceedings.mlr.press/v13/momma10a.html>.

- [11] John Platt. Sequential minimal optimization: A fast algorithm for training support vector machines. Technical Report MSR-TR-98-14, Microsoft, April 1998. URL: <https://www.microsoft.com/en-us/research/publication/sequential-minimal-optimization-a-fast-algorithm-for-training-support-vector-machines/>.
- [12] Sutharshan Rajasegarar, Christopher Leckie, James C. Bezdek, and Marimuthu Palaniswami. Centered hyperspherical and hyperellipsoidal one-class support vector machines for anomaly detection in sensor networks. *IEEE Transactions on Information Forensics and Security*, 5(3):518–533, 2010. doi:10.1109/TIFS.2010.2051543.
- [13] Bernhard Schölkopf, Robert C Williamson, Alex Smola, John Shawe-Taylor, and John Platt. Support vector method for novelty detection. In S. Solla, T. Leen, and K. Müller, editors, *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL: https://proceedings.neurips.cc/paper_files/paper/1999/file/8725fb777f25776ffa9076e44fcfd776-Paper.pdf.
- [14] Bernhard Schölkopf, John Platt, John Shawe-Taylor, Alexander Smola, and Robert Williamson. Estimating the support of a high-dimensional distribution. *Neural Computation*, 13:1443–1471, 07 2001. doi:10.1162/089976601750264965.
- [15] Juyang Weng, Yilu Zhang, and Wey-Shiuan Hwang. Candid covariance-free incremental principal component analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 25(8):1034–1040, 2003. doi:10.1109/TPAMI.2003.1217609.
- [16] Nurfazrina M. Zamry, Anazida Zainal, Murad A. Rassam, Eman H. Alkhamash, Fuad A. Ghaleb, and Faisal Saeed. Lightweight anomaly detection scheme using incremental principal component analysis and support vector machine. *Sensors*, 21(23), 2021. URL: <https://www.mdpi.com/1424-8220/21/23/8017>, doi:10.3390/s21238017.